FluxShard: Distributed Motion-Aware Cache Remapping for Real-Time Video Analytics

IEEE Publication Technology Department

***Abstract*—Edge-cloud video analytics caches intermediate features across consecutive frames to cut redundant computation and transmission. Existing methods operate at whole-scene granularity—reusing or invalidating entire feature maps—and assume a roughly stationary scene. Even moderate motion misaligns the cache with the current frame, triggering widespread invalidation although most content has merely shifted rather than truly changed. The root cause is a *granularity mismatch*: the cache is treated as an indivisible unit, yet real-world motion is local and heterogeneous.**

We present FluxShard, a motion-aware edge-cloud analytics system that elevates codec-level motion vectors (MVs) into a first-class control signal for feature caching. FluxShard decomposes the cached feature map into block-level shards that flow with observed motion, realigning reusable content and isolating genuinely changed regions as a minimal recomputation set. A *Receptive-Field Alignment Principle* guarantees bit-equivalent correctness of MV-guided reuse across convolutional layers, and a *profiling-driven truncation policy* sustains high sparsity under growing motion with negligible accuracy loss. At runtime, FluxShard adaptively routes the sparse residual workload between edge and cloud based on network conditions and device load. Evaluation on real-world dynamic video sequences spanning instance segmentation and pose estimation shows that FluxShard achieves up to 92% bandwidth reduction, 5.5× speedup, and ∼99% accuracy retention over state-of-the-art baselines.

***Index Terms*—Video analytics, robotics, edge-cloud collaborative system, CNN inference acceleration**

I. Introduction

Video analytics uses deep neural networks (DNNs) to process continuous video frames and produce results such as object detections, segmentation masks, and pose estimates. It is a core building block of intelligent edge applications: autonomous drones [1], [21], embodied robots [27], [10], augmented-reality headsets [28], and smart surveillance cameras [13] all rely on video analytics to perceive and react to their surroundings in real time. Running these models directly on the edge device avoids uploading raw video and keeps latency predictable, but edge hardware offers limited compute and energy. A common alternative is to offload frames to a powerful cloud server; however, streaming high-resolution video over a bandwidth-constrained wireless uplink introduces transmission delay that can easily exceed the real-time budget. Neither naive local nor remote processing reliably meets the joint latency and accuracy demands of these applications.

A natural way to reduce both transmission and computation is to exploit temporal redundancy between consecutive frames. In a video stream, most of the scene remains unchanged from one frame to the next, and this redundancy extends to the feature level: the intermediate feature maps produced by a DNN are also largely identical across consecutive frames. Reuse-based methods take advantage of this property by caching the feature maps of the previous frame and recomputing only the portions that have changed. Because the changed portion is typically small, both the data transmitted to the cloud and the computation performed on each frame are reduced simultaneously.

However, existing reuse methods are designed for fixed cameras and struggle with the complex motion patterns introduced by mobile edge devices. Some methods cache entire intermediate feature maps and reuse them only when consecutive frames are globally similar [17], [8]. Others compute per-pixel differences against the previous frame and recompute only the positions where the difference exceeds a threshold [2], [19], [24], [23]. Despite their different granularities, these methods share a common limitation: they treat the cached feature maps as an indivisible whole and cannot adapt to the diverse, non-uniform motion patterns in real-world scenes. On one hand, any camera ego-motion (the camera's own movement) or object movement triggers widespread cache invalidation, even when most content has merely shifted spatially rather than truly changed; this drives the reuse ratio close to zero and wastes both computation and bandwidth. On the other hand, when the global decision deems the cache "reusable," locally moving regions continue to use stale features, causing missed detections or localization errors. A cache management mechanism that can adapt to local, heterogeneous real-world motion is therefore essential for mobile video analytics.

Our key insight is that block-level motion vectors (MVs) from standard video codecs provide exactly such a motion signal that adapts to local, heterogeneous real-world motion. Specifically, the encoder partitions each frame into fixed-size blocks and records, for each block, the displacement direction and distance from the current frame to the previous frame. MVs offer two properties that make them well suited for cache management. First, they are a byproduct of the encoding process, so extracting them from an existing bitstream adds negligible cost. Second, because MVs capture geometric displacement rather than content change, they separate displacement-induced apparent change from regions that still require verification and recomputation. Re-indexing cached feature-map blocks along their MV trajectories therefore recovers reusable content that whole-scene methods would discard. Meanwhile, regions where MVs are inherently insufficient, such as non-rigid deformation, dis-occlusion, and newly appearing content, form the residual set.

Realizing this idea introduces two challenges. The first challenge is receptive field inconsistency. Reusing a cached

feature means reusing a previous computation result, which is only valid when the inputs to that computation are unchanged. In a DNN, each output position is computed from a spatial neighborhood of inputs called its receptive field. After MV-based alignment, heterogeneous motion can assemble a neighborhood whose positions originate from different motion regions, differing from the neighborhood used to produce the cached output. The reused value is then corrupted. Detecting such mismatches requires checking, for every output position at every layer, whether all inputs within its receptive field remain consistent. This verification cost is comparable to recomputation itself.

The second challenge is cache maintenance under heterogeneous motion. Existing methods update the feature cache in place: each new output simply overwrites the old value at the same grid location. This works when the camera is static, because scene content stays at roughly the same spatial location across frames. Under heterogeneous per-block motion, however, the cache reference frame itself drifts away from the current frame coordinate system. If one naively writes newly computed blocks back along their MVs, the mapping is many-to-one and incomplete: (i) multiple current blocks may target the same cache entry, creating write conflicts, and (ii) cache entries that no current block points to are never updated, becoming stale and not reusable. The cache therefore falls behind the evolving frame layout, hindering reuse in subsequent frames. A cache maintenance mechanism is needed to keep the cached feature maps aligned with each new frame.

We propose FluxShard, a motion-aware edge-cloud video analytics system that treats the feature cache as block-level shards flowing with motion vectors. FluxShard uses MVs as a first-class control signal to align cached features with the current frame, isolate the minimal residual set, and dynamically route work between local sparse update on the edge and selective offloading of only residual regions to the cloud, based on residual size, network conditions, and device load.

To address receptive field inconsistency, we observe that instead of checking whether the aligned input neighborhood is unchanged at every position of every layer, one can directly check whether the MVs within each receptive field are mutually consistent. If all positions in a receptive field share the same MV, the aligned neighborhood is identical to the original one, so the cached output is safe to reuse. FluxShard formalizes this observation as the Receptive Field Alignment Principle (RFAP) (§IV-B), a set of sufficient conditions that folds the per-layer, per-position neighborhood check into a single lightweight pass over the input-level MV field and identifies the positions that must be recomputed to keep MV-guided reuse correct under heterogeneous motion.

To address cache maintenance under heterogeneous motion, FluxShard borrows an insight from standard video decoding at the pixel level: a codec decoder reconstructs the current frame by warping reference blocks from the previous frame to their current positions using backward MVs, where each current position maps to exactly one source block, so the operation is conflict-free. FluxShard lifts this remapping from the pixel level to the feature level. After each frame, the remapped cache and the freshly computed residual outputs are fused into a new cache state expressed in the current-frame coordinate system. This remap-then-update rule preserves a valid reference without write conflicts or stale entries (§IV-C).

Grouped bars: edge-only latency vs. cloud RTT
at uplink = 5, 10, 20, 50 Mbps

Figure 1. Per-frame latency under varying uplink bandwidth. Neither pure-edge nor pure-cloud meets real-time across all conditions.

In summary, this paper makes the following contributions:

- We identify granularity mismatch as the key limitation of whole-scene cache reuse and show that codec-level motion vectors provide the right abstraction to overcome it.
- We propose FluxShard, a system that decomposes the feature cache into motion-aligned shards, combines sparse recomputation with motion-aware cache maintenance, and routes the residual workload between edge and cloud.
- We establish the Receptive Field Alignment Principle for bit-equivalent MV-guided feature reuse across layers, and an MV-based cache alignment scheme, inspired by codec decoding, that maintains spatial coherence under heterogeneous motion without write conflicts.
- We evaluate FluxShard on instance segmentation and human pose estimation with YOLOv11 [16] on two dynamic video benchmarks (DAVIS [25], 3DPW [26]). Compared to the strongest baseline, FluxShard achieves up to $5.5\times$ end-to-end speedup and 92% bandwidth reduction while retaining over 99% of full-model accuracy.

II. Challenges and Motivation

A. The Edge-Cloud Dilemma

Edge-cloud video analytics must navigate two competing bottlenecks. Running YOLOv11m [16] on an NVIDIA Jetson Xavier NX takes XX ms per 1080p frame, too slow for real-time applications. Offloading every frame removes the compute bottleneck but introduces a transmission one: JPEG-compressed 1080p at 30 fps sustains XX Mbps, routinely exceeding edge uplink capacity. Figure 1 quantifies this tension.

Feature cache reuse can reduce both bottlenecks: when consecutive frames share content, cached results substitute for fresh transmission and computation. Existing mechanisms [17], [8], [24], [23] still treat the cache in a fixed, whole-scene coordinate system, whether the decision is binary (reuse or recompute the entire frame) or pixel-level (propagate a dense difference map). This granularity mismatch breaks down under motion. Figure 2 shows that even modest displacement triggers widespread invalidation: the reuse ratio drops from XX% on near-static sequences to below XX% once motion exceeds XX px/frame, despite most content being merely shifted rather than truly changed.

Takeaway. Cache reuse is essential, but whole-scene granularity cannot tolerate motion. A finer-grained, motion-aware mechanism is needed.

Line plot: reuse ratio (%) vs. avg motion (px/frame) for a whole-scene cache baseline

Figure 2. Cache reuse ratio versus per-frame motion. Whole-scene granularity causes reuse to collapse under moderate motion.

Bars: residual set size w/o vs. w/ MV alignment across low / medium / high motion bins

Figure 3. MV alignment eliminates displacement-induced false misses, cutting the residual set by XX–XX%.

Bars: mAP for dense recomputation / whole-scene reuse / naïve MV reuse

Figure 4. Naïve MV-aligned reuse destroys accuracy due to receptive field inconsistency, even when MV alignment has already reduced the residual set.

Line plots: reuse ratio and accuracy relative to dense against frame index under naïve MV-guided write-back

Figure 5. Naïve MV-guided cache update causes progressive history drift over time. As stale and conflicting entries accumulate, the residual set grows, the reuse ratio falls, and accuracy relative to dense inference also declines.

B. Motion Vectors: Opportunity and Correctness Failure

H.264/H.265 codecs estimate a per-block displacement (d_x, d_y) as part of normal encoding, providing per-region motion information at zero additional cost. While recent studies [7], [6], [4] have shown that the codec's full reconstruction pipeline—transform, quantization, etc.—distorts feature statistics in ways that degrade downstream inference accuracy, the motion vectors themselves are a pure geometric signal untouched by these lossy stages. By shifting cached feature blocks according to these MVs, displaced content is realigned before differencing, and the remaining residual set concentrates on regions that MVs cannot explain: disocclusion, deformation, and newly appearing content. Figure 3 confirms MV alignment reduces the residual set by XX–XX% across motion intensities: most whole-scene cache misses stem from displacement, not true change.

This opportunity does not immediately translate into correct reuse, however. Despite the smaller residual set, directly reusing MV-aligned features produces severe accuracy loss. Figure 4 shows mAP drops from XX% to XX% under naïve MV reuse. The root cause is receptive field inconsistency: layers with receptive field size > 1 aggregate spatial neighborhoods. When adjacent blocks carry different MVs, re-indexing assembles patches that were never contiguous in the original frame; the cached output was computed from a different neighborhood. The resulting error compounds through cascaded layers.

Takeaway. MV alignment eliminates most false cache misses, but exploiting it requires a correctness guarantee under heterogeneous per-block motion. This is the first challenge our design must address.

C. Naïve Cache Update Breaks Future Reuse

Even with a correct residual set, the system still faces a second problem: how to maintain the cache after sparse recomputation. Existing reuse systems simply overwrite the cache in place. That policy assumes the cache and the current frame still share the same coordinate system. Under heterogeneous per-block motion, however, the cache remains anchored to the previous frame while the current content has moved to new locations.

A natural vanilla extension is therefore to write each newly recomputed residual block back to the cache location indicated by its MV. This update rule also fails because the mapping from current blocks to cached locations is many-to-one and incomplete. First, multiple current residual blocks can map to the same cached location, creating write conflicts and leaving the stored history path-dependent and ambiguous. Second, cached locations that no current block maps to are never refreshed, so stale content remains in the history. As a result, many positions that should remain reusable no longer find a valid history entry and are pushed into the residual set on the next frame. At the same time, positions that are declared reusable may read an incorrect historical value.

Once written into the history, these errors accumulate over time. Figure 5 tracks this failure over a representative sequence. With naïve MV-guided write-back, the reuse ratio steadily drops from XX% to XX%, because many genuinely reusable regions no longer align with a valid history entry and are pushed back into the residual set. At the same time, accuracy relative to dense inference falls from XX% to XX%, because nominally reusable positions increasingly consume corrupted cached features left by conflicting or missing updates.

Takeaway. Correct residual extraction is not enough. MV alignment can isolate a small residual set, but without first realigning the cache to the current frame layout, sparse updates leave behind stale or conflicting history. That broken history enlarges the next residual set and corrupts nominal reuse. Cache maintenance under heterogeneous motion is therefore the second challenge our design must address.

III. System Overview

This section presents the FluxShard system. We first define the system model and optimization objective (§III-A), then describe the end-to-end pipeline that addresses the two challenges identified above (§III-B).

A. System Model

We consider an edge-cloud video analytics system in which an edge device (e.g., an NVIDIA Jetson) receives a live video stream $\{I_t\}_{t=1}^{T}$ from a co-located camera, and a remote cloud server provides supplementary compute capacity. The two endpoints communicate over a bandwidth-limited uplink whose available bandwidth varies over time. Each endpoint hosts an identical copy of the inference model and maintains a per-layer feature cache of intermediate features from its

most recent inference. For each incoming frame I_t, the system makes a dispatch decision $d_t \in \{\text{edge}, \text{cloud}\}$, indicating whether to run inference locally at the edge or offload the frame to the cloud.

The deep neural network (DNN) $\mathcal{N}$ comprises L layers indexed by $l \in \{1, \ldots, L\}$. Layer l takes an input feature map $\mathbf{F}^l \in \mathbb{R}^{H'_l \times W'_l \times C'_l}$ and produces an output feature map $\mathbf{O}^l \in \mathbb{R}^{H_l \times W_l \times C_l}$; by convention $\mathbf{F}^1 = I_t$. Because our analysis concerns only spatial positions, we omit the channel dimension hereafter. The output at spatial position (i, j) depends on a set of input positions called its receptive field, denoted $\mathcal{R}^l(i, j) \subseteq \{1, \ldots, H'_l\} \times \{1, \ldots, W'_l\}$, with radius r_l. For convolutional and linear layers, $\mathbf{w}^l$ denotes the weight tensor applied over this receptive field.

Let $A_t(d_t)$ denote the task accuracy (e.g., mAP) achieved for frame I_t under the system behavior induced by dispatch decision d_t, and let A^*_t denote the accuracy of full recomputation without caching. Let $T_t(d_t)$ denote the corresponding per-frame end-to-end latency, comprising local computation, data transmission, and remote computation as applicable. We define the sequence-level averages $\bar{A} = \frac{1}{T}\sum_{t=1}^{T} A_t(d_t)$, $\bar{A}^* = \frac{1}{T}\sum_{t=1}^{T} A^*_t$, and $\bar{T} = \frac{1}{T}\sum_{t=1}^{T} T_t(d_t)$. Because caching-based acceleration trades accuracy for speed, the optimization objective balances the two:

$$\min_{\{d_t\}} \bar{T} \quad \text{s.t.} \quad \bar{A} \geq \alpha \bar{A}^*, \tag{1}$$

where $\alpha \in (0, 1]$ is a user-specified accuracy retention ratio.

We next define the cache and motion primitives used throughout the pipeline. Each endpoint caches intermediate features from its most recent inference throughout the DNN. When discussing a specific layer l, we write $\hat{\mathbf{F}}^l$ for the cached intermediate feature consumed by layer l and $\hat{\mathbf{O}}^l$ for the cached intermediate feature produced by layer l. The block-level motion vector (MV) field $\mathbf{m}_t$ is extracted from the video codec as a byproduct of decoding frame I_t; each entry maps a block position in I_t to its corresponding position in the reference frame. Given the MV field, a position (i, j) in the current frame maps to a cached position $(\hat{\imath}, \hat{\jmath})$; likewise, each input position $(p, q) \in \mathcal{R}^l(i, j)$ maps to its cached counterpart $(\hat{p}, \hat{q})$.

A shard is a block-sized unit of a feature map whose spatial extent corresponds to one entry in the MV field. At deeper layers with reduced spatial resolution, FluxShard uses downsampled layer-level MV fields measured in feature-map coordinates, denoted $\hat{\mathbf{m}}^l_{\text{in}}$ and $\hat{\mathbf{m}}^l_{\text{out}}$ on the input and output grids of layer l, so that shard alignment remains well defined throughout the network.

B. Pipeline

Figure 6 illustrates the end-to-end FluxShard pipeline. FluxShard introduces a dispatch layer as a virtual layer 0 that precedes the DNN. For each endpoint, this layer stores (i) a cached input and (ii) an accumulated MV field from that cached input to the present frame, enabling reusability estimation without invoking the DNN itself. The edge materializes the two endpoint-specific layer-0 states $(\hat{\mathbf{F}}^e_0, \hat{\mathbf{m}}^e_0)$ and

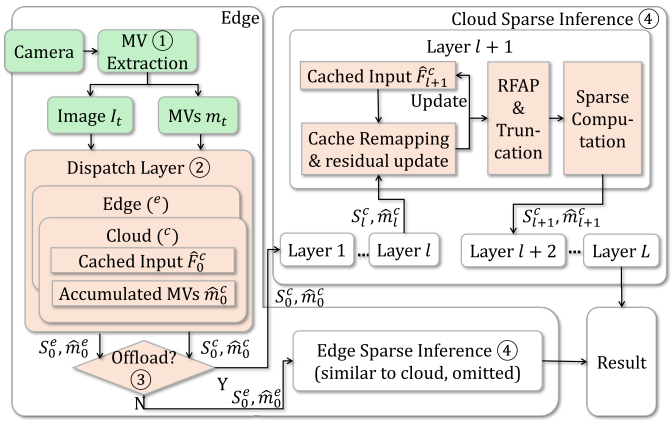

Figure 6. Overview of the FluxShard pipeline. The dispatch layer acts as a virtual layer 0 and stores the edge/cloud bookkeeping states $(\hat{\mathbf{F}}^e_0, \hat{\mathbf{m}}^e_0)$ and $(\hat{\mathbf{F}}^c_0, \hat{\mathbf{m}}^c_0)$, from which candidate input residual sets $\mathcal{S}^e_0$ and $\mathcal{S}^c_0$ are extracted. Layers labeled Layer 1 through Layer n in the figure denote the DNN layers of $\mathcal{N}$, indexed as $1, \ldots, L$ in the text. The cloud branch is expanded for illustration; after dispatch, sparse inference proceeds at only one selected endpoint.

$(\hat{\mathbf{F}}^c_0, \hat{\mathbf{m}}^c_0)$, which correspond to the edge and cloud endpoints, respectively.

When an encoded frame arrives at the edge, FluxShard proceeds in the following four stages.

Stage 1: MV extraction. Block-level motion vectors $\mathbf{m}_t$ are extracted from the video codec as a free byproduct of decoding, capturing per-region motion at no additional cost. Both dispatch states incorporate $\mathbf{m}_t$ into their respective accumulated MV fields, denoted $\hat{\mathbf{m}}^e_0$ and $\hat{\mathbf{m}}^c_0$ for the edge and cloud endpoints. When an endpoint performs inference, its accumulated field is reset; the other endpoint's field continues to accumulate, spanning a longer interval for future reuse.

Stage 2: Reusability estimation. Each dispatch state uses its accumulated MV field to shift its cached input, aligning displaced content with the current frame. A per-block comparison between the aligned cache and the decoded frame identifies positions with non-negligible difference; these positions form the candidate input residual sets $\mathcal{S}^e_0$ and $\mathcal{S}^c_0$. From their sizes, the edge state estimates local sparse inference latency, while the cloud state estimates the cost of transmitting the MV field and residual pixels plus remote inference under current network conditions.

Stage 3: Dispatch decision. The frame is assigned to the endpoint with the lower estimated latency, implicitly adapting to motion complexity, network bandwidth, and the execution cost at each endpoint.

Stage 4: Inference and cache update. After dispatch, only the selected endpoint continues sparse inference. We therefore omit endpoint superscripts below and write $\mathcal{S}_l$, $\hat{\mathbf{F}}^l$, and $\hat{\mathbf{O}}^l$ for the layerwise state of that selected endpoint. Figure 6 keeps the superscript c inside the zoomed branch only because it explicitly expands the cloud case. The selected endpoint begins from the layer-0 residual set and propagates it through the DNN one layer at a time. At layer l+1, FluxShard first remaps the cached input $\hat{\mathbf{F}}^{l+1}$ to the current-frame coordinate system and merges it with the output residuals from layer l to form

the updated input for computation. This updated input directly replaces the previous cache state, so the cache remains consistent with the current inference and serves as a valid reference for reuse in subsequent frames. RFAP (§IV-B) then identifies positions that remain safely reusable despite heterogeneous motion, and the truncation policy (§IV-D) further removes positions whose residual magnitude is sufficiently small. Only the remaining positions are sparsely recomputed. The resulting active positions define the output residual set passed to the next layer. The final result is produced by fusing reused and freshly computed features at the last layer. After inference, the selected endpoint stores the current frame as its layer-0 cached input and resets its accumulated MV field.

The non-selected endpoint retains its accumulated MV field, allowing its cache to span a longer interval for future reuse. When the server is selected, the edge transmits only $\hat{\mathbf{m}}_0^c$ and the residual pixels in $\mathcal{S}_0^c$.

IV. System Design

The pipeline in §III-B described the four stages that every frame traverses; this section develops the mechanisms deferred in that overview. We first formalize when a cached output of a layer can be directly reused after MV-guided shard alignment without accuracy loss (§IV-A); this criterion underpins all transmission and computation savings in FluxShard. We then derive the Receptive Field Alignment Principle (§IV-B), which makes the criterion efficiently evaluable under heterogeneous per-block motion by folding multi-layer consistency checks into a single pass over the input-level MV field. Next, we show how motion-aware cache remapping (§IV-C) maintains a valid current-frame reference after sparse updates. We then introduce a profiling-driven truncation policy (§IV-D) that assigns layer-specific thresholds to discard low-magnitude residuals with bounded task-level accuracy loss. Finally, we present the dispatch scheduler (§IV-E), which routes the residual workload between edge and cloud according to estimated end-to-end latency while satisfying the accuracy constraint in Eq. (1).

A. Layer-Local Reuse Criterion

Stage 4 of the pipeline (§III-B) propagates the residual set layer by layer, reusing cached outputs wherever possible. The quality of the final result hinges on a per-position decision: after MV-guided shard alignment, is the cached output close enough to the true output to be reused? This subsection formalizes that decision as a layer-local criterion and identifies the structural obstacle that heterogeneous MVs pose for its efficient evaluation.

a) Setup and notation: Three quantities coexist at every output position (i, j) of layer l:

- $\mathbf{O}^l(i, j)$: the *ground-truth* output, obtained by applying layer l to the current input $\mathbf{F}^l$;
- $\hat{\mathbf{O}}^l(\hat{\imath}, \hat{\jmath})$: the *cached* output from a previous frame, where $(\hat{\imath}, \hat{\jmath}) = (i, j) - \hat{\mathbf{m}}^l_{\text{out}}(i, j)$ is the MV-aligned source position;
- $\tilde{\mathbf{O}}^l(i, j)$: the *assembled* output actually consumed by layer $l{+}1$.

b) Reuse validity: Reuse at position (i, j) is *valid* when substituting the cached value for the ground truth introduces bounded error:

$$\left| \mathbf{O}^l(i, j) - \hat{\mathbf{O}}^l(\hat{\imath}, \hat{\jmath}) \right| \;\le\; \tau^l, \tag{2}$$

where τ^l is the tolerance assigned to layer l; setting $\tau^l = 0$ reduces the criterion to exact output equality. This condition is uniform across layer types and serves as the invariant maintained throughout the layer stack. In §IV-A and §IV-B, we focus on the exact case $\tau^l = 0$; §IV-D later reintroduces nonzero τ^l as a profiled approximation.

c) Layer output assembly: Based on this criterion, FluxShard assembles the output of layer l as

$$\tilde{\mathbf{O}}^l(i, j) = \begin{cases} \hat{\mathbf{O}}^l(\hat{\imath}, \hat{\jmath}), & \text{if Eq. (2) holds,} \\ \mathbf{O}^l(i, j), & \text{otherwise.} \end{cases} \tag{3}$$

The assembled output is thus a mosaic of *MV-aligned cached values* at reusable positions and *freshly computed values* at residual positions, and serves as the input to layer $l{+}1$. Eq. (2) is the semantic reuse criterion. The next paragraph converts it into an input-side sufficient condition, and the later subsections make that condition efficiently executable under heterogeneous motion.

d) Input-side sufficient condition: Eq. (2) cannot be checked directly without computing $\mathbf{O}^l(i, j)$, which defeats the purpose of reuse. Let $(\hat{p}, \hat{q}) = (p, q) - \hat{\mathbf{m}}^l_{\text{in}}(p, q)$ denote the MV-aligned source position in the cached input $\hat{\mathbf{F}}^l$ for each spatial location (p, q) in the current input $\mathbf{F}^l$, and let

$$\Delta^l_{\max}(i, j) \;=\; \max_{c,\, (p,q) \in \mathcal{R}^l(i,j)} \left| \mathbf{F}^l_c(p, q) - \hat{\mathbf{F}}^l_c(\hat{p}, \hat{q}) \right| \tag{4}$$

denote the maximum absolute input perturbation over all channels and spatial positions in the receptive field at output position (i, j). For a linear layer with weight vector $\mathbf{w}^l$ (the kernel flattened over all N input elements of the receptive field), linearity yields

$$\left| \mathbf{O}^l(i, j) - \hat{\mathbf{O}}^l(\hat{\imath}, \hat{\jmath}) \right| \;\le\; \|\mathbf{w}^l\|_1 \cdot \Delta^l_{\max}(i, j), \tag{5}$$

so Eq. (2) is guaranteed whenever the input patch satisfies

$$\Delta^l_{\max}(i, j) \;\le\; \frac{\tau^l}{\|\mathbf{w}^l\|_1}. \tag{6}$$

For nonlinear activations that are 1-Lipschitz (ReLU, sigmoid), the input-side bound directly implies the output-side one, so no extra margin is needed. Crucially, Eq. (6) uses only layer l's current input, cached input, and local weights; it never recurses to the model input.

In the exact setting, we use $\tau^l = 0$. In §IV-D, nonzero τ^l is introduced as a profiled approximation with the same normalized form.

This formulation avoids computing $\mathbf{O}^l$ itself, yet the check still inspects every position in the receptive field $\mathcal{R}^l(i, j)$, whose cost is comparable to executing the layer. To make the scheme practical, we next show how to infer these reuse decisions from a single pass over the input-level MV field, eliminating per-layer inspection entirely.

e) Residual set propagation: We denote by $\mathcal{S}_l$ the *residual set* at layer l: the set of output positions assigned to the "otherwise" branch of Eq. (3), where fresh computation is required. In this subsection, we analyze the exact residual set under $\tau^l = 0$; §IV-D later applies profiled nonzero thresholds that can only shrink $\mathcal{S}_l$. The initial residual set $\mathcal{S}_0$ is determined at the model input by comparing each MV-aligned position against its cached counterpart; subsequent layers inherit and potentially enlarge this set depending on their structure.

For a pointwise layer ($|\mathcal{R}^l| = 1$), each output depends on exactly one input position. Reused positions carry zero input difference and satisfy Eq. (6) with $\tau^l = 0$; residual positions carry nonzero difference and must be recomputed. The residual set therefore passes through unchanged: $\mathcal{S}_l = \mathcal{S}_{l-1}$.

Once $|\mathcal{R}^l| > 1$ (e.g., 3×3 convolution, pooling), the residual set can grow through two distinct mechanisms:

(a) MV inconsistency. Even when every input position in the receptive field is individually reusable (zero difference), heterogeneous MVs can relocate those positions from *disjoint* cached regions, assembling a spatial composition that never existed in the cached frame. The resulting mismatch violates Eq. (6) silently: the input-side check sees zero per-position difference yet the layer output diverges from its cached counterpart. As shown in Fig. 4, ignoring this inconsistency causes accuracy to degrade sharply with motion magnitude, dropping over XX% under moderate motion.

(b) Residual propagation. Each output position reads a spatial neighborhood. If any input position within that neighborhood belongs to $\mathcal{S}_{l-1}$, the output position may violate Eq. (6) and must join $\mathcal{S}_l$. Repeated application across cascaded spatial layers causes $\mathcal{S}_l$ to expand progressively, increasing the residual workload even under a uniform MV field.

Mechanism (a) is unique to heterogeneous per-block MVs. Delta methods [24], [9], [23] avoid it by restricting all blocks to a single shared MV, under which the spatial composition within every receptive field is guaranteed consistent; however, as shown in Fig. 3, this uniform-MV assumption collapses under heterogeneous motion. No prior work addresses the combination of per-block heterogeneous MVs with layer-wise feature cache reuse, where mechanism (a) is unavoidable. Naively, it can be resolved by checking MV consistency within every receptive field at every spatial aggregation layer, but the per-layer cost is comparable to recomputation itself, negating the benefit of reuse. §IV-B derives conditions that identify MV-inconsistent positions from a single pass over the input-level MV field, without restricting the MV field or inspecting every layer.

Mechanism (b) is an efficiency cost rather than a correctness failure. §IV-D mitigates it by filtering low-magnitude discrepancies via profiled layer-specific thresholds; the same mechanism also absorbs small residual discrepancies from mechanism (a).

B. Receptive Field Alignment Principle

The preceding subsection showed that spatial aggregation layers invalidate free propagation of the reuse map: each output position aggregates a neighborhood whose elements may originate from disjoint cached regions under heterogeneous motion. Checking the reuse criterion at every such layer costs as much as recomputation. This subsection derives the *Receptive Field Alignment Principle* (RFAP), which identifies two structural conditions on the input-level MV field that, when jointly satisfied, guarantee exact reuse at all downstream layers without per-layer inspection.

1) Two Conditions for Correct Reuse: The conflict above arises because each layer l carries two independent MV fields—$\hat{\mathbf{m}}^l_{\text{in}}$ on its input grid and $\hat{\mathbf{m}}^l_{\text{out}}$ on its output grid—and heterogeneous motion can make them geometrically incompatible. RFAP imposes two conditions that are jointly sufficient for reuse at any output position (i, j); violating either one can produce silent errors that accumulate across layers.

Condition 1: Intra-receptive-field uniformity. All input positions within the receptive field must share a single displacement:

$$\hat{\mathbf{m}}^l_{\text{in}}(p, q) = \hat{\mathbf{m}}^l_{\text{in}}(p', q'), \quad \forall\, (p, q),\, (p', q') \in \mathcal{R}^l(i, j). \quad (7)$$

A uniform shift preserves the relative arrangement of values consumed by the layer. If violated, the aligned patch stitches together fragments from non-contiguous cached regions—an input composition that never existed in the cached frame.

Condition 2: Input-output geometric coherence. Let $\pi^l_{p,q}(i, j) = s^l \cdot (i, j) + (p, q)$ map output position (i, j) to the input coordinate of receptive field entry (p, q), where s^l is the stride of layer l. When projected into input coordinates, the output-level displacement must land on the same cached position as the input-level displacement at every receptive field entry:

$$\pi^l_{p,q}\big((i, j) + \hat{\mathbf{m}}^l_{\text{out}}(i, j)\big) \;=\; \pi^l_{p,q}(i, j) \;+\; \hat{\mathbf{m}}^l_{\text{in}}\big(\pi^l_{p,q}(i, j)\big), \quad \forall\, (p, q) \in \mathcal{R}^l(i, j). \quad (8)$$

For the affine $\pi^l_{p,q}$ above, this simplifies to $\hat{\mathbf{m}}^l_{\text{out}} = \hat{\mathbf{m}}^l_{\text{in}}/s^l$, linking the two fields through the layer stride. If violated, the cached output and the aligned input patch refer to inconsistent cached content, as the two alignment paths diverge.

When both conditions hold, every input to the layer is identical between the current and cached evaluations. By determinism of the layer, $\mathbf{O}^l(i, j) = \hat{\mathbf{O}}^l(\hat{\imath}, \hat{\jmath})$, satisfying Eq. (2) with $\tau^l = 0$, i.e., exact reuse. Conversely, violating either condition can cause silent corruption that accumulates across layers. The set of output positions that fail either condition at layer l forms the *MV inconsistency mask* $\mathcal{M}^l$; these positions must be recomputed.

Recall that the reuse criterion in §IV-A already identifies positions that must be recomputed due to content change (the "otherwise" branch in Eq. (3)). $\mathcal{M}^l$ captures an orthogonal source of invalidity: positions where content may be unchanged yet the spatial arrangement within the receptive field differs from the cached evaluation. The union of both sets constitutes the complete per-layer residual set that is recomputed in Stage 4 of the pipeline; all remaining positions satisfy Eq. (2) and are safely assembled from cache. RFAP's role is therefore to make the reuse criterion of §IV-A enforceable under heterogeneous MV fields without per-layer

recomputation: it identifies, from the MV field alone, the minimal additional invalidations needed to keep the assembled output correct.

a) Practical relaxation: Checking and propagating $\mathcal{M}^l$ through every layer can inflate the residual set substantially under high motion. Note that when $\tau^l = 0$ for all l, applying RFAP only at the input level is equivalent to full per-layer RFAP: every inconsistent position produces an additional residual that propagates and is recomputed. With nonzero τ^l, the thresholds control how far each inconsistency propagates: positions whose activation change falls below τ^l are truncated, while those exceeding it remain in the residual set. There therefore exists a set of thresholds that preserves the accuracy of full RFAP while preventing $\mathcal{M}^l$ from inflating the residual set at deeper layers. FluxShard exploits this by checking RFAP only at the input level and calibrating $\{\tau^l\}$ via the profiling procedure in §IV-D to retain accuracy-critical residuals. §**??** confirms that this relaxation preserves accuracy while maintaining high sparsity under strong motion.

C. Motion-Aware Cache Remapping

RFAP determines which positions can be safely reused in the current frame, but it does not by itself specify how the cache should be stored for the next frame. Naïve in-place overwrite keeps the cache anchored to the previous coordinate system, while naïve MV-guided write-back creates the many-to-one conflicts and stale regions discussed in §II-C. Both break the reference frame assumed by Eqs. (2) and (6).

FluxShard resolves this issue with a remap-then-update rule. At layer $l+1$, it first remaps the cached input $\hat{\mathbf{F}}^{l+1}$ to the current-frame coordinate system and then merges it with the output residuals from layer l to form the updated input consumed by layer $l+1$, as described in Eq. (3). The resulting assembled tensor directly replaces the prior cached intermediate state. In layer-local notation, the same tensor can be viewed as the refreshed cached output of layer l and, equivalently, the refreshed cached input of layer $l+1$.

This update has two practical benefits. First, every current-frame position receives exactly one value, so the updated cache contains neither write conflicts nor unwritten holes. Second, the updated cache becomes the freshest reference for the next frame, because it is already expressed in the current frame coordinate system and incorporates the latest residual updates. This gives FluxShard the largest possible reuse opportunity at the next step under the maintained history. The same remap-then-update rule applies to the input cache and to all intermediate layer caches maintained by the sparse runtime.

D. Profiling-Driven Truncation Policy

Even with RFAP and cache remapping, exact reuse can still leave many low-magnitude residuals whose impact on the end task is negligible. FluxShard therefore adds a lightweight approximation, similar in spirit to DeltaCNN [24], that truncates small residuals at the dispatch layer and selected activation layers.

After RFAP determines the residual set $\mathcal{S}_l$, FluxShard applies truncation only within that set. A residual position is dropped and reused from cache whenever it satisfies the same input-side bound as Eq. (6), now with a nonzero tolerance τ^l. In practice, we profile nonzero thresholds only for the dispatch layer and activation layers; all other layers use $\tau^l = 0$ (thus their comparison is skipped). At the dispatch layer, the same rule uses the identity operator, i.e., $\|\mathbf{w}\|_1 = 1$, to form the input residual set $\mathcal{S}_0$.

Offline profiling jointly calibrates the dispatch-layer threshold τ^0 and the DNN thresholds $\boldsymbol{\tau} = \{\tau^l\}_{l=1}^{L}$ on a representative calibration set. Let $\mathcal{L}_{\mathrm{tr}} \subseteq \{1, \ldots, L\}$ denote the set of profiled DNN layers and let $K = |\mathcal{L}_{\mathrm{tr}}|$. We define the total admissible accuracy drop relative to the dense baseline as

$$\Delta A = (1 - \alpha) A^*. \tag{9}$$

Because the dispatch layer determines the input residual set and therefore all downstream workload, and because the input residual set directly determines the residual pixels sent when inference is offloaded, we reserve two thirds of this budget for τ^0 and split the remaining one third evenly across the profiled DNN layers:

$$b_0 = \frac{2}{3}\Delta A, \qquad b_l = \frac{1}{3K}\Delta A, \;\; l \in \mathcal{L}_{\mathrm{tr}}. \tag{10}$$

We then process the dispatch layer first and the profiled DNN layers in network order. For each profiled stage $u \in \{0\} \cup \mathcal{L}_{\mathrm{tr}}$, we search a discrete candidate set $\mathcal{C}^u$ and choose the largest threshold whose cumulative accuracy drop remains within the cumulative budget assigned up to that stage. This greedy search returns one calibrated τ^0 for input residual extraction and one τ^l for each profiled DNN layer.

At runtime, the dispatch layer applies τ^0 to form $\mathcal{S}_0$, and each profiled DNN layer applies its calibrated τ^l using the same criterion.

E. Dispatch Scheduler

The preceding subsections determine *what* to reuse and *what* to recompute; this subsection decides *where* the recomputation takes place. For each frame, the scheduler chooses between local sparse inference at the edge and sparse offload to the cloud to minimize end-to-end latency; the accuracy constraint is already enforced by the truncation policy.

a) Cost estimation: The edge maintains two dispatch-layer states, one tracking the local inference state and one mirroring the cloud state (§III). For each incoming frame, both states independently perform shard alignment and input residual extraction using their respective cached inputs and accumulated MV fields. Applying the dispatch-layer threshold τ^0 yields candidate residual sets $\mathcal{S}_0^e$ and $\mathcal{S}_0^c$. We denote their normalized sizes by $\rho_e = |\mathcal{S}_0^e|/|I_t|$ and $\rho_c = |\mathcal{S}_0^c|/|I_t|$.

We profile sparse inference latency offline on each endpoint as a function of this input residual ratio and store the result as lookup tables $\mathcal{T}_{\mathrm{edge}}$ and $\mathcal{T}_{\mathrm{cloud}}$. At runtime, edge latency is estimated as $T_{\mathrm{edge}} \leftarrow \mathcal{T}_{\mathrm{edge}}[\rho_e]$. Cloud latency combines the profiled inference cost with a transmission term: $T_{\mathrm{cloud}} \leftarrow \mathcal{T}_{\mathrm{cloud}}[\rho_c] + |\mathcal{S}_0^c|/\hat{B}$, where $\hat{B}$ is an exponentially weighted moving average of recent bandwidth measurements.

Algorithm 1: FluxShard Per-Frame Pipeline

Input: Current frame I_t; edge dispatch-layer state $(\hat{\mathbf{F}}_0^e, \hat{\mathbf{m}}_0^e)$; cloud dispatch-layer state $(\hat{\mathbf{F}}_0^c, \hat{\mathbf{m}}_0^c)$; profiled thresholds τ^0, $\boldsymbol{\tau}$, $\mathcal{T}_{\text{edge}}$, $\mathcal{T}_{\text{cloud}}$
Output: Inference result $\mathbf{y}_t$
Extract MV field $\mathbf{m}_t$ from codec for frame I_t;
$\hat{\mathbf{m}}_0^e \leftarrow \texttt{Accumulate}(\hat{\mathbf{m}}_0^e, \mathbf{m}_t)$;
$\hat{\mathbf{m}}_0^c \leftarrow \texttt{Accumulate}(\hat{\mathbf{m}}_0^c, \mathbf{m}_t)$;
$\mathcal{S}_0^e \leftarrow \texttt{ResidualSet}(I_t, \hat{\mathbf{F}}_0^e, \hat{\mathbf{m}}_0^e, \tau^0)$;
$\mathcal{S}_0^c \leftarrow \texttt{ResidualSet}(I_t, \hat{\mathbf{F}}_0^c, \hat{\mathbf{m}}_0^c, \tau^0)$;
$\rho_e \leftarrow |\mathcal{S}_0^e|/|I_t|$;
$\rho_c \leftarrow |\mathcal{S}_0^c|/|I_t|$;
$T_{\text{edge}} \leftarrow \mathcal{T}_{\text{edge}}[\rho_e]$;
$T_{\text{cloud}} \leftarrow \mathcal{T}_{\text{cloud}}[\rho_c] + |\mathcal{S}_0^c| \,/\, \hat{B}$;
if $T_{\text{edge}} < T_{\text{cloud}} - \epsilon$ **then**
  $\mathbf{y}_t \leftarrow \texttt{SparseInfer}_{\text{edge}}(I_t, \mathcal{S}_0^e, \boldsymbol{\tau}, \hat{\mathbf{m}}_0^e)$;
  Update edge cache state via remap-then-update; reset $\hat{\mathbf{m}}_0^e$;
else
  Transmit $\mathcal{S}_0^c$ and $\hat{\mathbf{m}}_0^c$ to cloud;
  $\mathbf{y}_t \leftarrow \texttt{SparseInfer}_{\text{cloud}}(\mathcal{S}_0^c, \boldsymbol{\tau}, \hat{\mathbf{m}}_0^c)$;
  Update cloud cache state via remap-then-update; reset $\hat{\mathbf{m}}_0^c$;
return $\mathbf{y}_t$;

The scheduler selects the endpoint with lower total latency; when the two estimates are within a margin ϵ, it prefers cloud offload to conserve edge energy.

Algorithm 1 summarizes the complete per-frame pipeline, integrating MV extraction, reusability estimation, dispatch, and cache maintenance.

V. Implementation

We prototype FluxShard on NVIDIA Jetson Xavier NX (edge) and NVIDIA RTX 3080 GPU (cloud), connected over a TCP link whose bandwidth is shaped with `tc` to emulate cellular uplink conditions.

a) Sparse inference engine: We store all feature tensors in channel-last (NHWC) layout and keep this layout throughout the sparse path. This matches FluxShard's per-position access pattern: MV-guided remapping and selective update touch spatial positions, while the full channel vector at each position remains contiguous. As a result, the sparse runtime avoids repeated layout conversion and does not introduce the strided gathers that a channel-first layout would incur.

We implement sparse convolution, pooling, and activation with custom CUDA kernels. The convolution kernel follows an implicit-GEMM structure with the same dense-style load/store pattern as its full counterpart, but skips arithmetic for inactive positions. At activation layers, truncation, MV-guided history lookup, and history refresh are fused into a single pass, so cache maintenance does not require a separate remapping kernel. On fully dense inputs, the custom convolution kernel reaches approximately XX% of cuDNN throughput; as sparsity increases, execution time decreases near-proportionally.

b) Offload metadata: FluxShard uses 16×16 MV blocks, so with two 16-bit components (int16) per block, the MV field occupies approximately 0.52% of the full RGB input in 8-bit format (uint8). For offloading, we further downsample the residual mask by 2×2 and pack it bitwise before transmission. The packed mask occupies approximately 1.04% of the input. Together, the MV field and mask contribute only 1.56% metadata overhead before the residual RGB pixels themselves are transmitted. These ratios are independent of the absolute input resolution.

VI. Evaluation

A. Experimental Setup

a) Testbed and network emulation: The edge device is an NVIDIA Jetson Xavier NX with 384 CUDA cores and 8 GB LPDDR4x memory. The cloud server hosts an NVIDIA RTX 3080 with 10 GB GDDR6X memory. Both machines run Ubuntu 20.04 with CUDA 11. They are connected through a 1 Gbps Ethernet switch, while the wireless uplink is emulated with Linux `tc`. We replay client-to-server bandwidth traces from a public 4G/5G measurement dataset [20] and group them into three tiers: *low*, *medium*, and *high*. The *low* tier uses 4G/LTE traces (40.4 ± 36.6 Mbps); the *medium* and *high* tiers split the 5G traces by median throughput into lower (382.8 ± 419.1 Mbps) and upper (596.9 ± 467.9 Mbps) halves. The emulated link also adds a fixed 20 ms one-way propagation delay. Unless otherwise noted, we report results under the medium tier.

b) Workloads and datasets: We evaluate two real-time dense prediction workloads with the YOLO11 family [16]: *instance segmentation* (Seg) with YOLO11m-seg on DAVIS [25], a video object segmentation benchmark containing 90 sequences (25–104 frames each) of diverse objects in open-world scenes with varied camera motion, and *pose estimation* (Pose) with YOLO11m-pose on 3DPW [26], an outdoor human activity dataset captured from a handheld camera, containing 61 sequences (273–2178 frames each). All inputs are resized to 1024×1024. We use the official checkpoints without fine-tuning. Table I summarizes the two workloads; dense local inference on the edge device is already impractical for real-time use, while pure cloud inference is fast but dominated by network transport. Our main evaluation focuses on these two dynamic-camera workloads. We reserve CMU Panoptic [15], a static-camera pose benchmark, for the ablation and alternative study.

c) Motion vectors: DAVIS and 3DPW are distributed as image sequences rather than compressed video. We therefore pre-compute one block-level motion-vector field for every consecutive frame pair at the same 16×16 granularity assumed by FluxShard. At runtime, the system consumes these pre-computed motion vectors exactly as it would consume codec-exported motion vectors in deployment. As shown in Table I, DAVIS exhibits substantially stronger motion than 3DPW (MV std 23.5 vs. 10.7 px), reflecting its diverse camera motion and object dynamics; this makes it a more challenging setting for cache reuse.

Table I
WORKLOAD PROFILE OF THE TWO EVALUATED TASKS. EDGE AND SERVER LATENCY ARE DENSE (NON-SPARSE) EXECUTION TIMES. EDGE ENERGY IS MEASURED PER FRAME VIA INA3221 POWER INTEGRATION. MV STD IS THE MEAN STANDARD DEVIATION OF BLOCK-LEVEL MOTION VECTORS ACROSS FRAMES, MEASURING MOTION INTENSITY.

	Seg (YOLO11m-seg)	Pose (YOLO11m-pose)
Dataset	DAVIS	3DPW
Input resolution	1024×1024	1024×1024
Parameters (M)	22.4	20.9
Edge latency (ms)	495.6 ± 25.9	444.5 ± 11.1
Edge energy (J)	6.62	6.79
Server latency (ms)	35.7 ± 3.0	27.6 ± 3.0
MV std (px)	23.5	10.7

d) Ground truth and metrics: For DAVIS segmentation, the dataset annotations do not directly match the deployed YOLO segmentation labels, so we generate pseudo ground truth with dense YOLO11x-seg inference on every frame. For 3DPW pose estimation, we use the dataset ground truth directly. We report mean Intersection over Union (mIoU) for segmentation, which measures the average overlap between predicted and ground-truth masks across all classes; higher is better. For pose estimation we report Object Keypoint Similarity (OKS), which measures the spatial agreement between predicted and ground-truth keypoints normalized by object scale; higher is better. End-to-end latency includes preprocessing, dispatch, transmission, inference, and cache maintenance. Edge energy is measured by integrating the Jetson Xavier NX on-board INA3221 `VDD_IN` power samples over each frame interval. Because FluxShard may execute a frame either locally or through offload, the reported energy is the realized per-frame average over the mixed online execution path.

e) Baselines: We compare FluxShard against four baselines. **Local** executes dense inference entirely on the edge device; because it is bandwidth-insensitive, we use it only as a workload reference rather than a main baseline. **Offload** sends every full frame to the cloud server for dense inference, representing the pure-offload upper bound on network cost. **COACH** [8] caches the entire previous output and reuses it when the whole-frame SSIM exceeds 0.95; when the frame must be transmitted, COACH quantizes the payload to 1/4 of its original size. It treats the cache as a single unit and cannot exploit partial reuse. **DeltaCNN** [24] propagates only the per-pixel difference between consecutive frames through the network, skipping positions whose activation change is below a fixed threshold; it maintains the feature cache in a fixed coordinate system without motion compensation. **MotionDeltaCNN** [23] (referred to as M-DeltaCNN) extends DeltaCNN by estimating a single global homography from the frame pair and shifting the entire cache accordingly before computing deltas; it still treats the cache as a rigid whole, so heterogeneous local motion is not handled at the shard level. Because MotionDeltaCNN is not open-sourced, we re-implement it on top of our own sparse execution engine. COACH, DeltaCNN and M-DeltaCNN share the same online dispatch logic as FluxShard: the client estimates edge-local latency and cloud-offload latency (transmission plus server inference) from profiled sparsity-ratio lookup tables, then routes each frame to whichever endpoint is faster. This ensures the comparison isolates reuse semantics rather than transport or scheduling differences. Unless stated otherwise, all reported statistics exclude the first frame of each video sequence, which performs full initialization and does not benefit from caching.

B. End-to-End Performance

a) Latency and energy under dynamic bandwidth: Figure 7 compares the realized online latency and edge energy across bandwidth tiers. FluxShard achieves the lowest latency and energy across both workloads and all tiers. Motion-aligned cache remapping keeps the residual set small, which simultaneously reduces the transmitted payload and the computation on both endpoints; the dispatcher then routes the smaller workload to whichever endpoint is cheaper.

For segmentation, FluxShard delivers 1.7–5.1×, 1.6–3.2×, and 1.5–2.7× latency reduction under low, medium, and high bandwidth respectively, with energy reduction of 1.6–2.8×, 1.4–1.8×, and 1.3–2.0×. Under low bandwidth, segmentation latency drops from 955 ms (pure offload) to 187 ms, and energy from 4.43 J to 1.59 J. For pose estimation, the gains are larger: 1.9–6.2×, 1.8–3.9×, and 1.5–2.9× latency reduction, with energy reduction of 1.6–2.4×, 1.4–2.5×, and 1.2–2.5×. Pose latency drops from 832 ms to 134 ms and energy from 3.77 J to 1.55 J under low bandwidth. The advantage is widest under low bandwidth, where transmission dominates and FluxShard's small residual payload cuts the bottleneck directly.

Among the reuse baselines, the relative ranking between COACH and the DeltaCNN variants shifts with bandwidth. Under low bandwidth, COACH's quantization compresses the full-frame payload enough to beat DeltaCNN on segmentation (312 ms vs. 525 ms), because DeltaCNN's sparsity gain and M-DeltaCNN's sparsity gain erode under high motion and its residual still saturates the link. Under medium and high bandwidth, M-DeltaCNN's motion-compensated sparsity helps it overtake COACH on both workloads, while DeltaCNN sometimes still trails COACH on segmentation where camera motion is more diverse. FluxShard outperforms all of them in every setting because motion-aligned cache remapping keeps the residual set consistently small, yielding both a smaller payload and less computation than any fixed-coordinate or whole-frame approach.

b) Accuracy: Table II reports accuracy across all settings. Our profiling-based configuration targets at most 3% accuracy loss relative to dense offload (the constraint $\bar{A} \geq \alpha \bar{A}^*$ in Eq. 1). FluxShard stays within 2.0–2.7% across both workloads and all tiers, meeting the budget with margin. DeltaCNN and M-DeltaCNN lose 1.6–2.9%, comparable to FluxShard. COACH incurs the largest drop at 5.4–7.2%: even with a high SSIM threshold (0.95), whole-frame similarity does not guarantee output-level fidelity, because small pixel-level changes can still affect dense prediction outputs such as segmentation masks and keypoint locations; quantization of the transmitted frames further compounds the loss. We note that COACH was originally designed for recognition tasks where the output is a

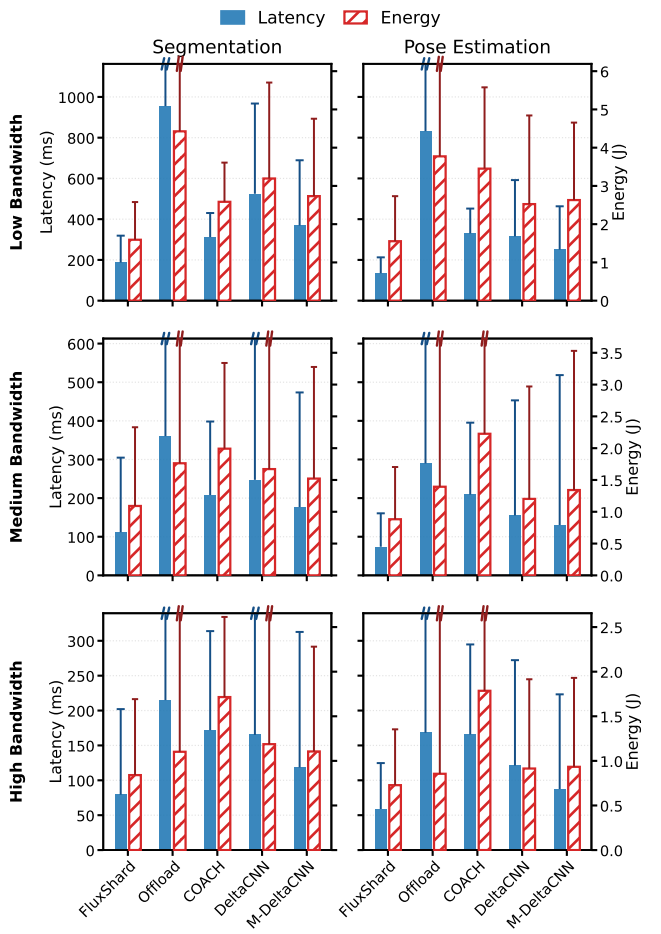

Figure 7. End-to-end latency (solid bars, left axis) and edge energy (hatched bars, right axis) under three bandwidth tiers. FluxShard reduces latency by up to 6.2× over pure offload and up to 2.8× over the best reuse baseline. Within each row, the two panels share the same axis range for direct comparison. Error bars show one standard deviation; break markers indicate clipped bars.

Table II
END-TO-END ACCURACY UNDER TRACE-DRIVEN BANDWIDTH REPLAY. PARENTHESIZED VALUES SHOW THE PERCENTAGE DROP RELATIVE TO DENSE OFFLOAD.

Workload	Method	Low	Medium	High
Seg (mIoU)	Offload	69.15	69.15	69.15
	FluxShard	67.75 (-2.02%)	67.71 (-2.08%)	67.30 (-2.68%)
	COACH	65.42 (-5.40%)	65.18 (-5.74%)	64.87 (-6.19%)
	DeltaCNN	68.06 (-1.58%)	68.07 (-1.57%)	67.99 (-1.68%)
	M-DeltaCNN	67.25 (-2.75%)	67.19 (-2.84%)	67.17 (-2.86%)
Pose (OKS)	Offload	70.95	70.95	70.95
	FluxShard	69.21 (-2.45%)	69.13 (-2.56%)	69.02 (-2.72%)
	COACH	67.14 (-5.37%)	66.19 (-6.71%)	65.85 (-7.19%)
	DeltaCNN	69.00 (-2.75%)	69.02 (-2.72%)	69.00 (-2.75%)
	M-DeltaCNN	69.22 (-2.44%)	69.03 (-2.71%)	68.93 (-2.84%)

single class label per frame, making it more tolerant of output-level variation than the dense prediction tasks evaluated here.

C. Microbenchmark

To understand where the end-to-end gains come from, we examine per-frame component statistics under medium bandwidth.

Table III
TRANSMISSION RATIO (FRACTION OF FULL FRAME TRANSMITTED), COMPUTATION RATIO (FRACTION OF DENSE FLOPS EXECUTED), AND CLOUD DISPATCH RATIO (FRACTION OF FRAMES ROUTED TO CLOUD) UNDER MEDIUM BANDWIDTH.

Method	Seg			Pose		
	Tx (%)	Comp (%)	Cloud (%)	Tx (%)	Comp (%)	Cloud (%)
FluxShard	17.1	47.9	98.1	8.9	40.7	95.6
Offload	100.0	100.0	100.0	100.0	100.0	100.0
COACH	24.0	96.0	97.5	24.1	96.5	93.4
DeltaCNN	56.9	66.8	98.4	31.7	54.6	96.4
M-DeltaCNN	45.5	65.7	98.0	28.9	58.5	93.7

a) Ratio breakdown: Table III reports transmission ratio (fraction of full frame transmitted), computation ratio (fraction of dense FLOPs executed), and cloud dispatch ratio (fraction of frames routed to cloud) under medium bandwidth for both workloads. Across all methods, segmentation transmission and computation ratios are consistently higher than pose, reflecting the stronger motion in DAVIS (MV std 23.5 vs. 10.7 px, Table I): more motion invalidates more cached positions, increasing both the transmitted payload and the computation.

FluxShard transmits only 8.9–17.1% of the full frame and executes 40.7–47.9% of dense FLOPs, compared with 28.9–56.9% and 54.6–66.8% for the DeltaCNN variants. COACH transmits 24% of the full frame thanks to its 4× quantization, but still computes over 96% of dense FLOPs because its whole-frame SSIM check rarely triggers reuse on dynamic video. In contrast, FluxShard achieves a lower transmission ratio while also cutting computation by more than half, with only 2.0–2.7% accuracy loss (Table II). The cloud dispatch ratio is high for all methods (93–100%), because the server is fast enough that offloading the sparse residual is almost always cheaper than local execution.

b) Latency breakdown: Figure 8 decomposes mean per-frame latency into preprocessing, transmission, and inference. The ratio differences in Table III translate directly into latency components. FluxShard has the smallest total bar on both workloads: its low transmission ratio and low computation ratio jointly shrink both the network and compute components. COACH's bar is dominated by transmission: although its payload is quantized to 24% of the full frame, it must recompute nearly all positions, so inference remains close to dense execution. COACH also shows a noticeable preprocessing cost because its whole-frame SSIM comparison is costly on the resource-constrained Jetson Xavier NX (the edge device). DeltaCNN and M-DeltaCNN reduce both transmission and inference through delta-based sparsity, but their fixed-coordinate caches leave a substantial active region under camera motion. Despite similar computation ratios (Table III), DeltaCNN and M-DeltaCNN show different inference latencies; we explain this gap in the sparsity-to-latency analysis below.

c) Sparsity-to-latency relationship: Figure 9 plots inference latency against compute ratio for pose estimation under medium bandwidth, split by dispatch path. The horizontal dashed line marks the dense inference latency on each endpoint. FluxShard and M-DeltaCNN both use our sparse

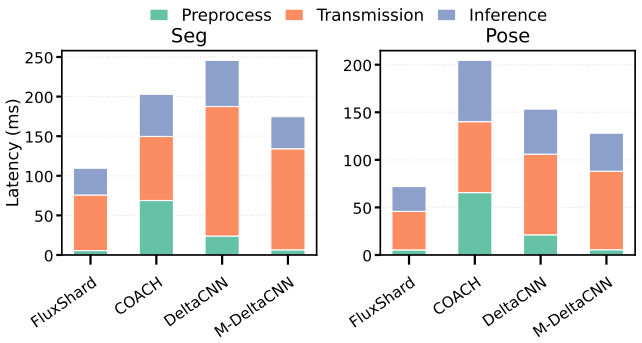

Figure 8. Mean per-frame latency breakdown under medium bandwidth. Preprocessing includes both edge-side and server-side preprocessing.

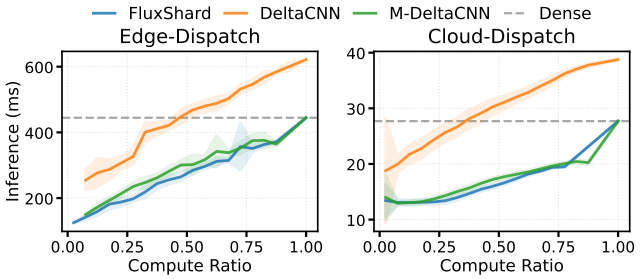

Figure 9. Inference latency vs. compute ratio for pose estimation under medium bandwidth, split by dispatch path. Dashed lines show dense inference latency on each endpoint. FluxShard and M-DeltaCNN share our sparse execution backend (overlapping trends); DeltaCNN uses its official CUDA implementation (different absolute level but similar slope).

execution backend, so their trends overlap closely; the latency difference between them comes from the compute ratio each method achieves, not from engine-level overhead. DeltaCNN uses its own official CUDA implementation, which follows a similar near-linear trend but at a different absolute level. This explains the latency gap between DeltaCNN and M-DeltaCNN observed in Figure 8: although their computation ratios are similar, the underlying engines differ. The near-linear relationship provides a strong guarantee that FluxShard's low compute ratio translates directly into low inference latency. This is particularly important under high bandwidth, where the transmission bottleneck is largely alleviated and the end-to-end advantage increasingly depends on reducing computation time.

VII. Related Work

Because consecutive video frames share substantial visual content, a growing body of work caches previously computed features and reuses them for subsequent frames, reducing both computation and transmission. Existing approaches fall into two broad categories.

Pipeline-level caching. Methods such as SPINN [17] and COACH [8] cache intermediate feature maps or label-level predictions and reuse them when successive inputs are deemed globally similar. COACH, for example, maintains semantic cluster centers in the feature space and triggers an early exit—bypassing cloud inference entirely when a new frame's embedding falls within a similarity threshold. These methods make a *binary, whole-input* reuse decision: either the entire cached result is reused or it is fully recomputed. This coarse granularity suits image classification, where a single label summarizes the scene, but cannot approximate the spatially dense outputs required by segmentation or detection.

Delta-based sparse inference. A second line of work including RRM [22], CBinfer [2], Skip-Convolution [9], and DeltaCNN [24] maintains a pixel-level reference cache and propagates only the *difference* (delta) between the current frame and the reference through the network, computing only at affected output locations and reusing cached values elsewhere. Among these, DeltaCNN represents the most complete realization: it achieves end-to-end sparse propagation with truncation buffers that prevent error accumulation, enabling unbounded-length sequences without dense resets and pushing the efficiency of static-camera settings to its practical limit. MotionDeltaCNN [23] extends DeltaCNN to moving cameras by warping the cache with a single global homography before computing the delta. However, a global transformation cannot capture locally heterogeneous motion—independently moving objects, depth-induced parallax, or mixed ego-motion and object motion—leaving large residual deltas that erode sparsity.

Shared limitation. Across both categories, the cache is treated as an *indivisible, scene-level entity*: it is either globally valid, globally stale, or aligned uniformly using a single transformation. Real-world mobile video, however, exhibits motion that is local and heterogeneous: a camera pan shifts the background uniformly while foreground objects move independently, and depth discontinuities create parallax that no single warp can reconcile. MotionDeltaCNN [23] reported that their homography alignment succeeds on only a small fraction of the DAVIS [25] sequences (14 out of 80) evaluated. The result is a *granularity mismatch*: the cache granularity is the whole scene, but motion granularity is per-region. This mismatch forces existing methods to either waste computation re-deriving content that has merely shifted, or sacrifice correctness by reusing misaligned features.

Note that several other techniques also reduce the cost of video analytics, including ROI filtering [3], [18], input resolution adaptation [14], [5], frame sampling that skips inference on selected frames [3], [18], and model compression via quantization or pruning [12], [11]. These strategies are orthogonal to feature cache reuse: they govern *which* frames, regions, or model to run, whereas caching determines whether to recompute or reuse at each spatial location within a given inference. FluxShard can be composed with any of them; this work focuses exclusively on the cache reuse axis.

VIII. Conclusion

We have presented FluxShard, a motion-aware video analytics system for real-time edge-cloud inference under dynamic motion. FluxShard treats granularity mismatch as the root cause of reuse failure in mobile video analytics and addresses it with motion-aligned shards, RFAP-based correctness checking, and motion-aware cache remapping.

Our edge-cloud implementation shows that this design preserves a small residual workload even when motion is local and heterogeneous, which reduces both transmission and computation. The resulting system achieves substantial bandwidth reduction and end-to-end speedup while maintaining strong task accuracy across diverse vision workloads. More broadly, FluxShard shows that lightweight codec motion signals can support correct and efficient cache reuse when correctness and cache maintenance are treated as first-class system problems.

REFERENCES

[1] Hassan J. Al Dawasari, Muhammad Bilal, Muhammad Moinuddin, Kamran Arshad, and Khaled Assaleh. Deepvision: Enhanced drone detection and recognition in visible imagery through deep learning networks. *Sensors*, 23(21), 2023.

[2] Lukas Cavigelli, Philippe Degen, and Luca Benini. CBinfer: Change-based inference for convolutional neural networks on video data.

[3] Tiffany Yu-Han Chen, Lenin Ravindranath, Shuo Deng, Paramvir Bahl, and Hari Balakrishnan. Glimpse: Continuous, real-time object recognition on mobile devices. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*, SenSys '15, pages 155–168. Association for Computing Machinery.

[4] Hyomin Choi and Ivan V. Bajić. Scalable image coding for humans and machines. 31:2739–2754.

[5] Kuntai Du, Qizheng Zhang, Anton Arapin, Haodong Wang, Zhengxu Xia, and Junchen Jiang. AccMPEG: Optimizing video encoding for video analytics.

[6] Lingyu Duan, Jiaying Liu, Wenhan Yang, Tiejun Huang, and Wen Gao. Video coding for machines: A paradigm of collaborative compression and intelligent analytics. 29:8680–8695.

[7] Leonardo Galteri, Lorenzo Seidenari, Marco Bertini, and Alberto Del Bimbo. Deep generative adversarial compression artifact removal. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 4836–4845. ISSN: 2380-7504.

[8] Luyao Gao, Jianchun Liu, Hongli Xu, Sun Xu, Qianpiao Ma, and Liusheng Huang. Accelerating end-cloud collaborative inference via near bubble-free pipeline optimization. *arXiv preprint arXiv:2501.12388*, 2024.

[9] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. Skip-convolutions for efficient video processing. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2694–2703. ISSN: 2575-7075.

[10] Beining Han, Meenal Parakh, Derek Geng, Jack A. Defay, Gan Luyang, and Jia Deng. FetchBench: A simulation benchmark for robot fetching.

[11] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1*, volume 1 of *NIPS'15*, pages 1135–1143. MIT Press.

[12] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2704–2713. IEEE.

[13] Fatemeh Jalali, Morteza Khademi, Abbas Ebrahimi Moghadam, and Hadi Sadoghi Yazdi. Robust scene aware multi-object tracking for surveillance videos. 638:130114.

[14] Junchen Jiang, Ganesh Ananthanarayanan, Peter Bodik, Siddhartha Sen, and Ion Stoica. Chameleon: scalable adaptation of video analytics. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '18, pages 253–266. Association for Computing Machinery.

[15] Hanbyul Joo, Tomas Simon, Xulong Li, Hao Liu, Lei Tan, Lin Gui, Sean Banerjee, Timothy Scott Godisart, Bart Nabbe, Iain Matthews, Takeo Kanade, Shohei Nobuhara, and Yaser Sheikh. Panoptic studio: A massively multiview system for social interaction capture. 2017.

[16] Rahima Khanam and Muhammad Hussain. Yolov11: An overview of the key architectural enhancements. *arXiv preprint arXiv:2410.17725*, 2024.

[17] Stefanos Laskaridis, Stylianos I Venieris, Mario Almeida, Ilias Leontiadis, and Nicholas D Lane. Spinn: synergistic progressive inference of neural networks over device and cloud. In *Proceedings of the 26th annual international conference on mobile computing and networking*, pages 1–15, 2020.

[18] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. Reducto: On-camera filtering for resource-efficient real-time video analytics. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, SIGCOMM '20, pages 359–376. Association for Computing Machinery.

[19] Mostafa Mahmoud, Kevin Siu, and Andreas Moshovos. Diffy: a déjà vu-free differential deep neural network accelerator. In *Proceedings of the 51st Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO-51, pages 134–147. IEEE Press.

[20] Arvind Narayanan, Xumiao Zhang, Ruiyang Zhu, Ahmad Hassan, Shuowei Jin, Xiao Zhu, Xiaoxuan Zhang, Denis Rybkin, Zhengxuan Yang, Zhuoqing Morley Mao, Feng Qian, and Zhi-Li Zhang. A variegated look at 5g in the wild: performance, power, and QoE implications. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, SIGCOMM '21, pages 610–625. Association for Computing Machinery.

[21] Kien Nguyen, Feng Liu, Clinton Fookes, Sridha Sridharan, Xiaoming Liu, and Arun Ross. Person recognition in aerial surveillance: A decade survey. pages 1–1.

[22] Bowen Pan, Wuwei Lin, Xiaolin Fang, Chaoqin Huang, Bolei Zhou, and Cewu Lu. Recurrent residual module for fast inference in videos. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1536–1545. ISSN: 2575-7075.

[23] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. MotionDeltaCNN: Sparse CNN inference of frame differences in moving camera videos with spherical buffers and padded convolutions. pages 17292–17301.

[24] Mathias Parger, Chengcheng Tang, Christopher D Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. Deltacnn: End-to-end cnn inference of sparse frame differences in videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12497–12506, 2022.

[25] Jordi Pont-Tuset, Federico Perazzi, Sergi Caelles, Pablo Arbeláez, Alexander Sorkine-Hornung, and Luc Van Gool. The 2017 davis challenge on video object segmentation. *arXiv:1704.00675*, 2017.

[26] Timo von Marcard, Roberto Henschel, Michael Black, Bodo Rosenhahn, and Gerard Pons-Moll. Recovering accurate 3d human pose in the wild using imus and a moving camera. In *European Conference on Computer Vision (ECCV)*, sep 2018.

[27] Qi Wu, Zipeng Fu, Xuxin Cheng, Xiaolong Wang, and Chelsea Finn. Helpful DoggyBot: Open-world object fetching using legged robots and vision-language models.

[28] Jun Zhang, Mina Henein, Robert Mahony, and Viorela Ila. VDO-SLAM: A visual dynamic object-aware SLAM system.

A. Biographies and Author Photos

```
\begin{IEEEbiographynophoto}{Jane Doe}
Biography text here without a photo.
\end{IEEEbiographynophoto}
```

or a biography with a photo

```
\begin{IEEEbiography}[{\includegraphics
[width=1in,height=1.25in,clip,
keepaspectratio]{fig1.png}}]
{IEEE Publications Technology Team}
In this paragraph you can place
your educational, professional background
and research and other interests.
\end{IEEEbiography}
```

Please see the end of this document to see the output of these coding examples.

Jane Doe Biography text here without a photo.

IEEE Publications Technology Team In this paragraph you can place your educational, professional background and research and other interests.